Ca' Foscari University of Venice

Software Architectures

Software architectures
Pietro Ferrara
pietro.ferrara@unive.it

Ca' Foscari University of Venice

Save the date! 11/10/2025 Google DevFest @ Campus

- Organized already last year and the year before
  - https://devfest25.gdgvenezia.it/
  - https://devfest24.gdgvenezia.it/
  - https://devfest23.gdgvenezia.it/
  - https://gdg.community.dev/events/details/google-gdg-venezia-presents-devfest-triveneto-2022/
- 16 enterprises will be at the event
  - Open for interviews for stages, jobs, etc etc…
- Already 350 attendees registered in the platform
  - And more than 200 meetings with enterprises booked
- Hurry up!!!

Ca' Foscari University of Venice

Eiffel tower - from PO1

- 18,038 metallic parts
- 5,300 workshop drawings
- 50 engineers and designers
- 150 workers in the Levallois-Perret factory
- Between 150 and 300 workers on the construction site
- 2,500,000 rivets
- 7,300 tonnes of iron
- 60 tonnes of paint
- 2 years, 2 months and 5 days of construction
- History here

La tour Eiffel prise du Champ-de-Mars - Exposition universelle de 1889
© Parisienne de photographie - Neurdein / Roger-Viollet
https://artsandculture.google.com/asset/_/hQEdYmo3Gzetsw

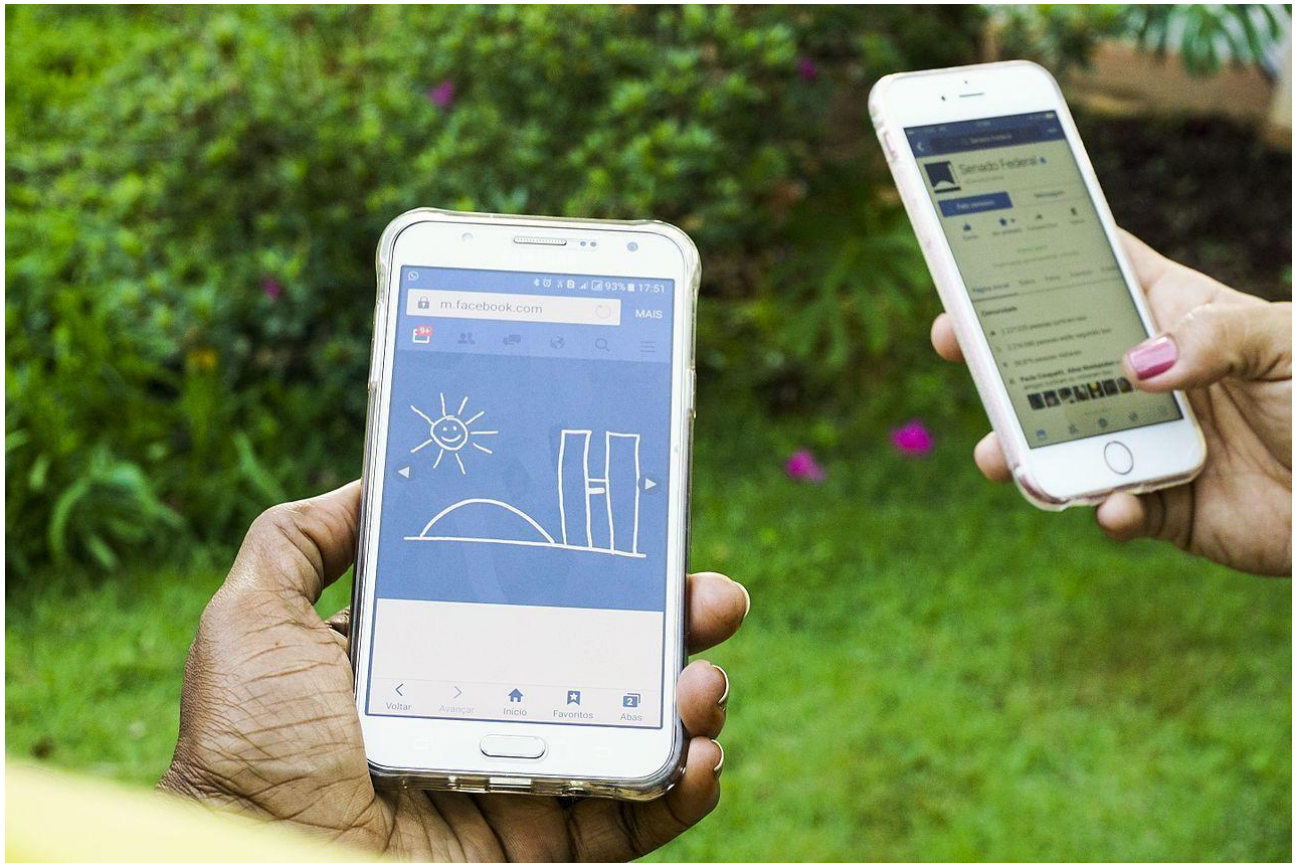
Engineering - from PO1

- Most of the tower was built somewhere else from some workers
- On the site pieces were only assembled, not constructed!
  - With first temporary rivets and then assembled rivets
  - But still, only a third of the rivets were used on site!
- About 500 workers involved in the construction with different skills
- They "communicated" through some standard means
  - Such as rivets, drawings, etc…
- <u>A worker using some results of other works did not need to know the details of what the others have done!</u>

Ca' Foscari University of Venice

Smartphone (just as an example) - from PO1

- … what do we mean by smart?
- Android operating system
  - https://source.android.com/
  - https://github.com/aosp-mirror
- 99 GitHub repositories
- "Main" repository
  - ~ 280 branches
  - > 30K commits
  - Almost 150 contributors
  - > 10MLOCs



By Senado Federal - Fotos produzidas pelo Senado, CC BY 2.0, https://commons.wikimedia.org/w/index.php?curid=53990377

Technologies

- https://www.youtube.com/watch?v=QgnJ8GpsBG8
  - Forrest Gump, 1994

- I'll feel often like Forrest Gump while running
  - Teaching technologies that I don't master
    - Indeed, often just with an high level idea of them!
  - To master a technology one needs months/years of experience
    - See job announcements
- But please… never be like the people following Forrest Gump!
  - Neither professors nor colleagues nor "experts" nor …
  - Always question yourself if this technology is what you need and why

Ca' Foscari University of Venice

A typical Eiffel tower for sw developers

how to supervise the construction of the tower
<u>Software architectures</u>, Development Methodologies, …

how to build up more complex systems packaging and assembling basic blocks
Software engineering, Project management, Cloud computing, Web application …

how to build the basic blocks of the tower
Programming, Computer architecture, Object-oriented programming, Databases, Operating Systems

Ca' Foscari University of Venice

Software developers career

CTOs

Software architects

Project managers

Software engineers

Software developers

Content of the course

- Software architecture is a very practical topic
  - People tried many solutions
  - They started to copy each other
  - The most effective solutions took the lead
- Not always clear why a solution was preferred
  - Or better, a lot of speculation and different opinions
- Always think with your brain!
- The main goals of the course are to provide you
  - An overview of the main architectures adopted in the history
  - Critical thinking about architectures
- In 5 years software architectures will be different from today!

Ca' Foscari University of Venice

What is software architecture?

*"Software architecture refers to the **fundamental structures of a software system** and the discipline of creating such structures and systems. (...) Software architecture is about making **fundamental structural choices** that are **costly to change** once implemented."*

WIKIPEDIA
The Free Encyclopedia

- High level thinking about software systems
  - Not about programming languages, type of databases, etc…
- Make decisions before writing code (design)
  - That cannot be easily changed later on

Concept vs technology

- Concept: abstract ideas that can be applied to different contexts and are not bound to specific situations
- Technology: the application of concepts for practical purposes, especially in industry
- University is about teaching concepts
- Enterprise is about developing and applying technologies
- Concepts are better understood and remembered if practiced
  - We need (some) technology to exemplify concepts
  - But which technology? Why?
- Lectures will be split between concepts and technologies

Ca' Foscari University of Venice

Blended course

- Technologies will be covered with online material
  - ~10-15 minutes youtube video
    - Introducing the main points of the technology and connecting it with the conceptual part of the course
  - HTML pages with guided tutorials
  - Exercises at the end of the tutorials
    - To be done BEFORE moving to the next online lectures
- 12 online lectures on Friday 3:45-5:15
  - No need to be connected or whatever
  - I'll be always available by email to answer any questions

Ca' Foscari University of Venice

Course outline

- 10/11 in-presence lectures on concepts
- 12 online lectures on technologies
- 1/2 lectures on practical experience (with invited speakers)
  - Dec. 15th: Alberto Pietrogrande, IT Architect Director Tinexta Cyber
    - https://www.linkedin.com/in/alberto-pietrogrande/
    - My former boss
  - Gianluca Caiazza, Secura Factors, Zamperla project (maybe)

Choosing a technology

- https://www.google.com/search?q=popularity+of+technologies+for+developing+microservices
- A lot of confusion, nothing really widely accepted
- Already from a programming language point of view
  - Java, Python, node.JS, …
- Need to make a completely arbitrary choice!
  - Is it the right one? The best one? … who knows!
- We'll go with Java and Spring (and some other technologies)
  - But feel free to experiment with others!!!
    - Especially if you are already familiar with other technologies

Course outline

- Fourth year I am teaching this course
  - Second year blended
- Topics might change during the course for different reasons
  - Maybe I'll be slower or faster than expected
  - Other topics might arise while teaching the lectures
  - Feel free to propose topics!
- We'll combine theory and practice
  - Theory: frontal lectures aligned with the textbook
  - Practice (a kind of lab): mostly coding and tutorials using different technologies
    - Online!

Course outline

Software architecture characteristics:

- Modularity and coupling
- Operational characteristics: availability, performance, scalability, recoverability
- Structural characteristics: deployability, configurability, extensibility, upgradeability
- Measurement of characteristics

Course outline

Architectural patterns (tentative list):

- Monolithic vs. distributed architectures
- Ball of Mud
- Layered
- Pipeline
- Microkernel
- Service-based
- Event-driven
- Space based
- Microservice

Course outline

Technologies (tentative list):

- Docker[compose]: from artifact to [virtual] machines/containers
- Gradle: from code to artifact
- Spring [Web]: hosted artifacts
- Hibernate: connecting code to data
- Camel: pipeline architecture
- REST: communications between distributed components
- RabbitMQ: asynchronous communication through messages
- Kubernetes: automatic replication and orchestration of machines
- AWS Lambda (?): serverless computing

Textbook

- Mark Richards, Neal Ford: "Fundamentals of Software Architecture",2nd edition, O'Reilly Media, March 2025
  - http://fundamentalsofsoftwarearchitecture.com/ (1st edition)
  - Only about concepts (no code!)
  - 1st edition available in BAS, asked for 2nd edition
- Additional material: Chris Richardson, "Microservices patterns: with examples in Java"
  - Available in the library (BAS)
  - Not needed, just if you want to deepen some topics for your interests
  - But this contains code!!!
  - And a lot of more details about popular architectural patterns

(mostly) covered

added this year

Preface

1. Introduction
- Defining Software Architecture
- Laws of Software Architecture
- Expectations of an Architect
  - Make Architecture Decisions
  - Continually Analyze the Architecture
  - Keep Current with Latest Trends
  - Ensure Compliance with Decisions
  - Understand Diverse Technologies
  - Know the Business Domain
  - Possess Interpersonal Skills
  - Understand and Navigate Politics
- Roadmap

Part I. Foundations

2. Architectural Thinking
- Architecture Versus Design
  - Strategic Versus Tactical Decisions
  - Level of Effort
  - The Significance of Trade-Offs
- Technical Breadth
  - The 20-Minute Rule
  - Developing a Personal Radar
- Analyzing Trade-Offs
- Understanding Business Drivers
- Balancing Architecture and Hands-On Coding
- There's More to Architectural Thinking

3. Modularity
- Modularity Versus Granularity
- Defining Modularity
- Measuring Modularity
  - Cohesion
  - Coupling
  - Core Metrics
  - Distance from the Main Sequence
  - Connascence
- From Modules to Components

4. Architectural Characteristics Defined
- Architectural Characteristics and System Design
- Architectural Characteristics (Partially) Listed
  - Operational Architectural Characteristics
  - Structural Architectural Characteristics
  - Cloud Characteristics
  - Cross-Cutting Architectural Characteristics
- Trade-Offs and Least Worst Architecture

5. Identifying Architectural Characteristics
- Extracting Architectural Characteristics from Domain Concerns
- Composite Architectural Characteristics
- Extracting Architectural Characteristics
  - Working with Katas
- Kata: Silicon Sandwiches
  - Explicit Characteristics
  - Implicit Characteristics
- Limiting and Prioritizing Architectural Characteristics

6. Measuring and Governing Architecture Characteristics
- Measuring Architecture Characteristics
  - Operational Measures
  - Structural Measures
  - Process Measures
- Governance and Fitness Functions
  - Governing Architecture Characteristics
  - Fitness Functions

The Scope of Architectural Characteristics
- Architectural Quanta and Granularity
- Synchronous Communication
- The Impact of Scoping
  - Scoping and Architectural Style
  - Kata: Going Green
- Scoping and the Cloud

Component-Based Thinking
- Defining Logical Components
- Logical Versus Physical Architecture
- Creating a Logical Architecture
  - Identifying Core Components
  - Assigning User Stories to Components
  - Analyzing Roles and Responsibilities
  - Analyzing Architectural Characteristics
  - Restructuring Components
- Component Coupling
  - Static Coupling
  - Temporal Coupling
  - The Law of Demeter
- Case Study: Going, Going, Gone—Discovering Components

Architecture Partitioning
Kata: Silicon Sandwiches—Partitioning
Monolithic Versus Distributed Architectures
Fallacy #1: The Network Is Reliable
Fallacy #2: Latency Is Zero
Fallacy #3: Bandwidth Is Infinite
Fallacy #4: The Network Is Secure
Fallacy #5: The Topology Never Changes
Fallacy #6: There Is Only One Administrator
Fallacy #7: Transport Cost Is Zero
Fallacy #8: The Network Is Homogeneous
The Other Fallacies
Team Topologies and Architecture
On to Specific Styles

10. Layered Architecture Style
Topology
Style Specifics
Layers of Isolation
Adding Layers
Data Topologies
Cloud Considerations
Common Risks
Governance
Team Topology Considerations
Style Characteristics
When to Use
When Not to Use
Examples and Use Cases

11. The Modular Monolith Architecture Style
Topology
Style Specifics
Monolithic Structure
Modular Structure
Module Communication
Data Topologies
Cloud Considerations
Common Risks
Governance
Team Topology Considerations
Style Characteristics
When to Use
When Not to Use
Examples and Use Cases

12. Pipeline Architecture Style
Topology
Style Specifics
Filters
Pipes
Data Topologies
Cloud Considerations
Common Risks
Governance
Team Topology Considerations
Style Characteristics
When to Use
When Not to Use
Examples and Use Cases

13. Microkernel Architecture Style
Topology
Style Specifics
Core System
Plug-In Components
The Spectrum of "Microkern-ality"
Registry
Contracts
Data Topologies
Cloud Considerations
Common Risks
Volatile Core
Plug-In Dependencies
Governance
Team Topology Considerations
Architecture Characteristics Ratings
Examples and Use Cases

Part II. Architecture Styles

9. Foundations
Styles Versus Patterns
Fundamental Patterns
Big Ball of Mud
Unitary Architecture
Client/Server

14. Service-Based Architecture Style
Topology
Style Specifics
Service Design and Granularity
User Interface Options
API Gateway Options
Data Topologies
Cloud Considerations
Common Risks
Governance
Team Topology Considerations
Style Characteristics
Examples and Use Cases

15. Event-Driven Architecture Style
Topology
Style Specifics
Events Versus Messages
Derived Events
Triggering Extensible Events
Asynchronous Capabilities
Broadcast Capabilities
Event Payload
The Swarm of Gnats Antipattern
Error Handling
Preventing Data Loss
Request-Reply Processing
Mediated Event-Driven Architecture
Data Topologies
Monolithic Database Topology
Domain Database Topology
Dedicated Data Topology
Cloud Considerations
Common Risks
Governance
Team Topology Considerations
Style Characteristics
Choosing Between Request-Based and Event-Based Models
Examples and Use Cases

16. Space-Based Architecture Style
Topology
Style Specifics
Processing Unit
Virtualized Middleware
Messaging Grid
Data Grid
Processing Grid
Deployment Manager
Data Pumps
Data Writers
Data Readers
Data Topologies
Cloud Considerations
Common Risks
Frequent Reads from the Database
Data Synchronization and Consistency
High Data Volumes
Data Collisions
Governance
Team Topology Considerations
Style Characteristics
Examples and Use Cases
Concert Ticketing System
Online Auction System

17. Orchestration-Driven Service-Oriented Architecture
Topology
Style Specifics
Taxonomy
Reuse…and Coupling
Data Topologies
Cloud Considerations
Common Risks
Governance
Team Topology Considerations
Style Characteristics
Examples and Use Cases

18. Microservices Architecture
Topology
Style Specifics
Bounded Context
Granularity
Data Isolation
API Layer
Operational Reuse
Frontends
Communication
Choreography and Orchestration
Transactions and Sagas
Data Topologies
Cloud Considerations
Common Risks
Governance
Team Topology Considerations
Style Characteristics
Examples and Use Cases

19. Choosing the Appropriate Architecture Style
Shifting "Fashion" in Architecture
Decision Criteria
Monolith Case Study: Silicon Sandwiches
Modular Monolith
Microkernel
Distributed Case Study: Going, Going, Gone

20. Architectural Patterns
Reuse
Separating Domain and Operational Coupling
Communication
Orchestration Versus Choreography
CQRS
Infrastructure
Broker-Domain Pattern

Part III. Techniques and Soft Skills

21. Architectural Decisions
- Architectural Decision Antipatterns
  - The Covering Your Assets Antipattern
  - Groundhog Day Antipattern
  - Email-Driven Architecture Antipattern
- Architectural Significance
- Architectural Decision Records
  - Basic Structure
  - Example
  - Storing ADRs
  - ADRs as Documentation
  - Using ADRs for Standards
  - Using ADRs with Existing Systems
  - Leveraging Generative AI and LLMs in Architectural Decisions

22. Analyzing Architecture Risk
- Risk Matrix
- Risk Assessments
- Risk Storming
  - Phase 1: Identification
  - Phase 2: Consensus
  - Phase 3: Risk Mitigation
- User-Story Risk Analysis
- Risk-Storming Use Case
  - Availability
  - Elasticity
  - Security
- Summary

23. Diagramming Architecture
- Diagramming
  - Tools
  - Diagramming Standards: UML, C4, and ArchiMate
  - Diagram Guidelines
- Summary

24. Making Teams Effective
- Collaboration
- Constraints and Boundaries
- Architect Personalities
  - The Control-Freak Architect
  - The Armchair Architect
  - The Effective Architect
- How Much Involvement?
- Team Warning Signs
  - Process Loss
  - Pluralistic Ignorance
- Leveraging Checklists
  - Developer Code-Completion Checklist
  - Unit and Functional Testing Checklist
  - Software-Release Checklist
- Providing Guidance
- Summary

25. Negotiation and Leadership Skills
- Negotiation and Facilitation
  - Negotiating with Business Stakeholders
  - Negotiating with Other Architects
  - Negotiating with Developers
- The Software Architect as a Leader
  - The 4 Cs of Architecture
  - Be Pragmatic, Yet Visionary
  - Leading Teams by Example
- Integrating with the Development Team
- Summary

26. Architectural Intersections
- Architecture and Implementation
  - Operational Concerns
  - Structural Integrity
  - Architectural Constraints
- Architecture and Infrastructure
- Architecture and Data Topologies
  - Database Topology
  - Architectural Characteristics
  - Data Structure
  - Read/Write Priority
- Architecture and Engineering Practices
- Architecture and Team Topologies
- Architecture and Systems Integration
- Architecture and the Enterprise
- Architecture and the Business Environment
- Architecture and Generative AI
  - Incorporating Generative AI into Architecture
  - Generative AI as an Architect Assistant
- Summary

27. The Laws of Software Architecture, Revisited
- First Law: Everything in Software Architecture Is a Trade-Off
  - Shared Library Versus Shared Service
  - Synchronous Versus Asynchronous Messaging
  - First Corollary: Missing Trade-Offs
  - Second Corollary: You Can't Do It Just Once
- Second Law: Why Is More Important Than How
  - Out of Context Antipattern
- The Spectrum Between Extremes
- Parting Words of Advice

We won't cover part 3!!!

Ca' Foscari University of Venice

[Approximative] History

Technologies

PCs

Widespread of Internet

Cloud computing

Spring

RabbitMQ/Kafka

REST APIs

Kubernetes

AWS Lambda

Software architectures

Plugin

Model-View-Controller

Service-oriented

Layered architecture

Serverless computing

Microservices

Modular monoli

1980 1990 2000 2010 2020

Ca' Foscari University of Venice

My [precise] history

Microsoft

Unsafe C# code

Aug-Nov 2007: intern

Thesis about Java multithreading

PhD in CS

Thesis about JavaCard

Master in CS

Bachelor in CS

OO programming and Scala

Postdoc and lecturer

ETH zürich

Android and JavaScript

Research Staff Member

IBM

.NET and Java

Head of R&D

JULIASOFT code analysis reinvented

Assistant professor

Associate professor

2000 2003 2005 2009 2013 2016 2020 2023

Ca' Foscari University of Venice

Communicating with the professor

- Moodle Forum
  - Then the same forum, then the same forum…
- Obviously, you are still free to send me emails!
  - But please use the forum if it is a generic questions about the content of the course, sw architectures, etc..
  - And for questions about exams, grades, etc.. please always give a look to these slides before asking, usually the answer is already here
- E-mail address: pietro.ferrara@unive.it
- Linkedin: https://www.linkedin.com/in/pietroferrara/
  - Feel free to connect with me!

No tutor this year

- Because of budget cuts, blended courses won't have a tutor available [starting from] this year
  - This has been always the case in the last years
- Therefore:
  - No Telegram group (indeed not much used last year)
  - I'll provide support on the online material
    - But apologize me if I'll be slow in answering…

Your wooclap poll will be displayed here

1

Install the **Chrome** or **Firefox extension**

2

Make sure you are in **presentation mode**

wooclap

https://app.wooclap.com/events/SADM (questions from 1 to 7)

Ca' Foscari University of Venice

Project and written exam (1/2)

- 3 group tasks during the course (two thirds of the grade)
  - 2 small ones at the beginning
    - To define the context and characteristics of the software architecture
  - 1 big task at the end implementing the architecture
    - Using any technology, not only the ones I'll introduce you
  - Part time students can instead submit a unique final project
  - Groups composed by 3 to 5 students, tell me the composition in the forum
- Topic: implement an IT system to <do something>
  - Still under discussion
  - It might be used as a baseline for projects of other courses!

Project and written exam (2/2)

- You will have to deliver at the end of the project
  - A GitHub repository containing the code of your software (architecture)
  - The history of the repo should make clear the contribution of the different students in the group
  - It must contain a description (pdf, markdown, …) of the structure of the architecture with references to the code
- Written exam at the end of the course (one third of the grade)
  - **Only after submitting and passing** the tasks/project
  - Example from last year in Moodle
  - About **only all** the in presence lectures

Ca' Foscari University of Venice

Software Architecture + Development Methodologies

- Some of you need to take also the Development Methodologies exams
  - Two modules of a 12 ECTS course
- In this case
  - You need to pass <u>both</u> the modules in the <u>same academic year</u>
    - i.e., if you pass one module but not the other one you'll need to retake the full exam next year
  - The final grade will be the average of the two grades
- The written exam of the two modules will be at the same time
  - Split in two sessions with a short break in the middle to allow people that take only one module to enter and exit the exam room

Ca' Foscari University of Venice

Additional material

- We must provide additional material to some students
  - Working, with difficulties in learning, etc.
  - See here https://www.unive.it/pag/42819/
- As additional material, those students will have access to the recording of my lectures in the Moodle space
  - Note that I cannot decide who is allowed to have access
  - In case you think you should have access but you don't have it, contact directly Ca' Foscari offices
  - If you are an international student, usually you lose access to the recordings once you arrive in Italy
- Indeed, registrations open to everyone (1st sem. 1st year MsC)

Ca' Foscari University of Venice

Calendar

- All in presence lectures will take place in Aula A
- Quite irregular schedule
  - First 2 weeks, 4 in presence lectures
  - Third week: 2 online lectures
    - https://www.dagstuhl.de/seminars/seminar-calendar/seminar-details/25421
  - After that
    - In presence lectures on Monday
    - Online lectures on Thursday
  - Last online lecture will be in January
    - But I'll publish it before mid December

Technologies

- We will use various technologies (in addition to the ones aforementioned)
  - IDE (IntelliJ IDEA in particular)
  - Java
  - Git[Hub]
    - Premium account of GitHub for free for students and professors, https://education.github.com/benefits
  - Maven
  - Stackoverflow ☺️ or ChatGPT?
  - And much more TBD